

Sviluppo di Interfacce Web con Flask

Pagine statiche, DOM, template dinamici ed eventi

UCBM – Ingegneria Industriale

A.A. 2025/2026

- Capire che cosa vede il browser quando apriamo una pagina web
- Introdurre HTML come struttura di una pagina statica
- Introdurre CSS come descrizione dell'aspetto visuale
- Capire il DOM come albero di oggetti della pagina
- Passare gradualmente da pagine statiche a pagine generate con Python
- Leggere route Flask, template Jinja, form ed eventi

Punto di partenza

- Abbiamo già una base dati palestra
- Abbiamo già una REPL applicativa
- Abbiamo una versione web Flask
- Ora studiamo la parte di interfaccia, partendo dalle basi

Scelta didattica

Prima costruiamo l'idea di pagina statica. Solo dopo introduciamo elementi dinamici prodotti da Python.

REPL

- menu testuale
- input da tastiera
- output su terminale
- flusso sequenziale

Web

- pagine
- link
- form
- navigazione tra viste

Che cosa resta uguale

- Il dominio: iscritti, istruttori, esercizi, schede, esecuzioni
- Le operazioni CRUD
- I vincoli di integrità
- Il database come stato persistente
- Il bisogno di trasformare input utente in operazioni sui dati

Che cosa cambia

- Il modo in cui l'utente sceglie un'operazione
- Il modo in cui i dati vengono raccolti
- Il modo in cui i risultati vengono mostrati
- Il ciclo tecnico: richiesta HTTP, risposta HTML
- La presenza del browser come parte del sistema

- Il browser è il programma che visualizza la pagina
- Riceve documenti HTML dal server
- Applica regole CSS per costruire l'aspetto grafico
- Costruisce una rappresentazione interna chiamata DOM
- Gestisce interazioni come click, input e invio di form

Definizione

Una pagina statica è una pagina il cui contenuto è scritto direttamente nel file HTML.

- Non dipende dal database
- Non cambia in base all'utente
- Non contiene dati calcolati da Python
- È il punto più semplice per capire struttura e DOM

Una pagina HTML minima

```
<!doctype html>
<html lang="it">
  <head>
    <title>Pagina demo</title>
  </head>
  <body>
    <h1>Benvenuti</h1>
    <p>Questa è una pagina statica.</p>
  </body>
</html>
```

Come leggere la pagina minima

- `html`: radice del documento
- `head`: informazioni sulla pagina
- `title`: titolo mostrato dal browser
- `body`: contenuto visibile
- `h1`: titolo principale
- `p`: paragrafo di testo

Tag, elemento, attributo

- Un tag indica il tipo di oggetto: `p`, `a`, `section`
- Un elemento è il tag più il suo contenuto
- Un attributo aggiunge informazioni all'elemento
- Gli attributi spesso controllano destinazioni, nomi, classi e comportamenti

Esempio di attributo

```
<a href="/admin">Amministratore</a>
```

- a: elemento link
- href: destinazione del link
- Amministratore: testo visibile nella pagina

Oggetto visibile	HTML	Idea
Titolo	<code><h1></code>	intestazione principale
Testo	<code><p></code>	paragrafo
Area logica	<code><section></code>	gruppo di contenuti
Link	<code><a></code>	navigazione
Immagine	<code></code>	risorsa visuale

- HTML deve prima descrivere il significato degli oggetti
- Il titolo è un titolo, non solo testo grande
- Un link è una scelta di navigazione
- Una sezione raggruppa contenuti collegati
- Questa struttura diventa leggibile anche dal browser

CSS: a che cosa serve

Definizione

CSS descrive come gli elementi HTML devono apparire.

- Colori
- Spaziature
- Bordi
- Dimensioni
- Disposizione degli elementi nella pagina

Una regola CSS semplice

```
.panel {  
  border-radius: 16px;  
  padding: 1.5rem;  
}
```

- `.panel`: seleziona elementi con classe `panel`
- `border-radius`: arrotonda gli angoli
- `padding`: aggiunge spazio interno


```
<section class="panel">  
  <h2>Scheda forza base</h2>  
  <p>Allenamento gambe e petto.</p>  
</section>
```

- L'attributo class collega HTML e CSS
- Il CSS seleziona .panel
- Il browser applica lo stile all'elemento

Definizione

DOM significa Document Object Model: è la rappresentazione ad albero della pagina costruita dal browser.

- Il browser non lavora solo con testo HTML
- Trasforma i tag in oggetti
- Organizza gli oggetti come nodi collegati
- Questa struttura è ciò con cui l'utente interagisce

Da HTML ad albero DOM

```
<section class="hero">  
  <h1>Applicazione palestra</h1>  
  <a href="/admin">Amministratore</a>  
</section>
```

```
document  
  html  
    body  
      section.hero  
        h1  
        a
```

Perché il DOM è importante

- L'utente non interagisce con il file HTML astratto
- Interagisce con oggetti costruiti nel browser
- Un click avviene su un nodo del DOM
- Un valore digitato vive in un nodo `input`
- Un form inviato raccoglie valori dai suoi campi

Interfaccia come mappa di oggetti

Oggetto	Nodo DOM	Azione possibile
Pannello	<code>section</code>	leggere contenuti
Link	<code>a</code>	navigare
Campo	<code>input</code>	inserire un valore
Pulsante	<code>button</code>	attivare un'azione
Form	<code>form</code>	inviare dati

Definizione operativa

Un evento è qualcosa che accade durante l'interazione: click, submit, modifica di un campo, caricamento della pagina.

- Alcuni eventi restano nel browser
- Alcuni eventi generano richieste HTTP
- Flask vede solo la richiesta arrivata al server

Evento: click su un link

click su "Amministratore"

- > browser legge href

- > richiesta GET /admin

- > il server produce una nuova pagina

Lettura

Prima di parlare di Flask, il punto è capire che il click su un link chiede un'altra pagina.

Seconda idea: pagina dinamica

Definizione

Una pagina dinamica è una pagina il cui HTML finale viene costruito dal server usando dati e codice.

- Il browser riceve comunque HTML
- Ma quell'HTML non è scritto tutto a mano
- Python può scegliere dati, valori e righe da mostrare
- Il database può contribuire al contenuto della pagina

Da pagina statica a pagina generata

HTML statico:

```
<h1>Benvenuti</h1>
```

HTML generato:

```
<h1>{{ titolo }}</h1>
```

- Nel primo caso il testo è fisso
- Nel secondo caso il testo arriva da Python

- Client: il browser dell'utente
- Server: il programma Flask
- Il browser non esegue il codice Python dell'applicazione
- Il browser invia richieste HTTP
- Flask risponde con HTML, redirect o errori

Ciclo richiesta-risposta

utente

- > browser

- > richiesta HTTP

- > route Flask

- > dati Python / query / logica

- > template HTML

- > risposta HTTP

- > DOM nel browser

- Flask è un micro-framework web per Python
- Riceve richieste HTTP
- Associa URL a funzioni Python
- Produce risposte
- Integra template HTML tramite Jinja

Una route Flask

```
@app.get("/admin")
def admin_home():
    return render_template(
        "role.html",
        role="amministratore"
    )
```

- /admin è l'URL
- admin_home è la funzione Python
- role.html è il template della risposta

- Un template è HTML con parti variabili
- Flask passa dati Python al template
- Jinja produce HTML finale
- Il browser riceve HTML, non codice Python

```
<h1>{{ role }}</h1>
```

```
render_template("role.html", role="istruttore")
```

Risultato

Il DOM viene costruito a partire dall'HTML finale, non dal template sorgente.

```
{% for row in rows %}  
  <tr>  
    <td>{{ row.nome }}</td>  
    <td>{{ row.cognome }}</td>  
  </tr>  
{% endfor %}
```

- Una lista Python diventa una tabella HTML
- Ogni record può generare una riga visibile

- Molte pagine condividono intestazione e navigazione
- `layout.html` definisce la struttura base
- Le pagine specifiche riempiono un blocco
- Questo riduce duplicazione e rende più coerente l'interfaccia

Estendere un layout

```
{% extends "layout.html" %}

{% block content %}
    <section class="hero">
        ...
    </section>
{% endblock %}
```

GET

- legge una pagina
- non dovrebbe modificare dati
- può essere ripetuta

POST

- invia dati
- può modificare lo stato
- richiede maggiore attenzione

- Un form raccoglie valori dall'utente
- Ogni campo ha un attributo `name`
- Il nome del campo diventa chiave in `request.form`
- Il metodo decide come i dati vengono inviati

Da input a request.form

```
<input name="nome">
```

```
nome = request.form["nome"].strip()
```

Mappa mentale

Oggetto grafico -> valore nel browser -> richiesta HTTP -> dato letto da Flask.

Evento: submit di un form

submit del form nuovo iscritto

- > POST /entity/iscritti/new

- > Flask legge request.form

- > INSERT nel database

- > redirect alla lista

- Il browser può fare controlli minimi
- Flask deve comunque controllare ciò che riceve
- Il database applica vincoli finali
- Gli errori devono tornare all'utente in forma comprensibile

- Il POST modifica lo stato
- Dopo il POST, Flask esegue un redirect
- Il browser fa una nuova GET
- Evitiamo reinvii accidentali aggiornando la pagina

Flash message

```
flash("Record aggiornato.", "success")  
return redirect(url_for("entity_list", name=name))
```

- Il messaggio informa l'utente
- Il redirect porta alla pagina aggiornata

Dal database al DOM

- 1 Query SQL
- 2 Righe Python
- 3 Template Jinja
- 4 HTML finale
- 5 DOM nel browser
- 6 Oggetto grafico visibile

Esempio: statistiche home

```
stats = {  
    "iscritti": conta("Iscritti"),  
    "istruttori": conta("Istruttori"),  
    "schede": conta("SchedeAllenamento"),  
    "esecuzioni": conta("Esecuzioni"),  
}
```

Template della home

```
<div class="stats">
  <div><strong>{{ stats.iscritti }}</strong><span>iscritti</span></div>
  <div><strong>{{ stats.istruttori }}</strong><span>istruttori</span></div>
</div>
```

Lettura

Il numero viene dal database, ma l'oggetto visibile nasce nel template.

- L'applicazione palestra usa route generiche per alcune CRUD
- Una configurazione descrive tabella, chiave, colonne e campi
- La stessa funzione può servire iscritti, istruttori ed esercizi

```
@app.get("/entity/<name>")
def entity_list(name):
    config = entity_config(name)
    ...
```

- /entity/iscritti passa name = iscritti
- /entity/istruttori passa name = istruttori
- Flask mappa una parte dell'URL a una variabile Python

Operazione	HTTP	Esempio
Lista	GET	/entity/iscritti
Nuovo form	GET	/entity/iscritti/new
Crea	POST	/entity/iscritti/new
Modifica form	GET	/entity/iscritti/1/edit
Aggiorna	POST	/entity/iscritti/1/edit
Elimina	POST	/entity/iscritti/1/delete

JavaScript: dopo DOM e form

- JavaScript reagisce a eventi nel browser
- Può modificare il DOM
- Può inviare richieste senza ricaricare tutta la pagina
- Non è indispensabile per una prima applicazione Flask

Progressione

Prima capiamo pagine statiche, DOM, route e form. Poi JavaScript diventa più leggibile.

Leggere un click nell'applicazione palestra

- 1 Aprire la home
- 2 Individuare il link nel template
- 3 Leggere l'URL generato
- 4 Trovare la route in `app.py`
- 5 Trovare il template restituito
- 6 Osservare il DOM finale nel browser

Leggere un form nell'applicazione palestra

- 1 Aprire il form
- 2 Individuare i campi e i loro name
- 3 Capire se il metodo è GET o POST
- 4 Trovare la route che elabora il POST
- 5 Seguire `request.form`
- 6 Seguire la query SQL
- 7 Seguire redirect e messaggio

- Il link punta a una route inesistente
- Il template usa una variabile non passata da Flask
- Il form usa un `name` diverso da quello letto nel codice
- La query viola un vincolo del database
- Il CSS seleziona una classe non presente nel DOM

Dove cercare un problema

Sintomo	Primo punto da controllare
Pagina non trovata	route e URL
Dati mancanti	query e parametri passati al template
Form non letto	attributi <code>name</code> e <code>request.form</code>
Errore di vincolo	schema e chiavi esterne
Pagina visualmente errata	classi CSS e struttura DOM

- ➊ Avviare le demo Flask progressive
- ➋ Osservare una pagina statica
- ➌ Collegare HTML, CSS e DOM
- ➍ Passare a template e dati Python
- ➎ Aprire l'applicazione palestra
- ➏ Seguire click, route, form e query

- Cartella: `labs/flask_interfacce_demo`
- Sei demo piccole e indipendenti
- Ogni demo isola un concetto dell'interfaccia
- Poi lo stesso concetto viene ritrovato nell'applicazione palestra

- ➊ Pagina statica: HTML e DOM
- ➋ Template dinamico: dati Python e Jinja
- ➌ Componenti grafici: oggetti visibili e tag
- ➍ Form GET/POST: submit e `request.form`
- ➎ Eventi DOM: click e input lato browser
- ➏ Mini CRUD: Post/Redirect/Get

Avvio delle demo

```
python3 -m pip install -r labs/flask_interfacce_demo/requirements.txt
```

```
python3 labs/flask_interfacce_demo/app.py
```

```
http://127.0.0.1:5000
```


Avvio dell'applicazione palestra

```
python3 labs/palestra_web/app.py --reset
```

```
http://127.0.0.1:5000
```

Obiettivo

Non usare l'applicazione solo come utente finale: osservare il percorso tra pagina, codice e database.

- Scegliere una operazione: creare esercizio, creare iscritto, registrare esecuzione
- Disegnare la catena: oggetto grafico, evento, richiesta, route, query, risposta
- Individuare i file coinvolti
- Spiegare dove si potrebbe inserire una validazione

- Una base dati non basta: serve progettare come l'utente interagisce
- Le pagine devono corrispondere a casi d'uso
- I form devono produrre dati coerenti con lo schema
- Gli eventi devono invocare operazioni applicative chiare
- Il progetto deve essere spiegabile prima di essere implementato

- Una pagina statica è il primo livello da capire
- HTML descrive la struttura
- CSS descrive l'aspetto
- Il DOM è l'albero di oggetti della pagina nel browser
- Flask usa Python per produrre HTML dinamico
- I form collegano interfaccia e operazioni applicative

- Leggere le demo Flask in ordine
- Riconoscere gli stessi concetti nell'applicazione palestra
- Modificare una vista semplice
- Aggiungere una piccola validazione
- Progettare pagine e route per un nuovo dominio d'esame